

這份 **AngularJS** 程式的教學手冊，幫助你了解整體架構與各個功能的運作方式。

Category			
Title			
Description			
price			
Image Path			
Submit			

Title	Price	Image	Actions
Fjallraven - Foldsack No. 1 Backpack, Fits 15 Laptops	109.95		Edit Remove
Mens Casual Premium Slim Fit T-Shirts	22.3		Edit Remove
Mens Cotton Jacket	55.99		Edit Remove
Mens Casual Slim Fit	15.99		Edit Remove
John Hardy Women's Legends Naga Gold & Silver Dragon Station Chain Bracelet	695		Edit Remove

[Previous](#) Page 1 of 4 [Next](#)

■ 範例名稱：產品管理系統（Product Management）

本教學使用 **AngularJS** 搭配 **RESTful API**（[Fake Store API](#)）實作基本的「商品管理」功能，包括：

- 讀取產品列表（GET）
- 分頁功能
- 新增產品（POST）
- 編輯產品（PUT）
- 刪除產品（DELETE）

🧠 第一部分：AngularJS App 註冊與 Controller 初始化

```
var app = angular.module("ProductManagement", []);
```

```
app.controller("ProductManagementController", fetchProducts);
```

- 宣告 **AngularJS** 模組 **ProductManagement**
- 註冊 **ProductManagementController** 並指定邏輯由 **fetchProducts** 函式處理

🧠 第二部分：Controller 的主體 - fetchProducts

這是整體應用的核心控制邏輯。

```
function fetchProducts($scope, $http) {
```

- 使用 **\$scope** 綁定資料與 **UI** 元素
- 使用 **\$http** 與後端 **API** 溝通

🚩 初始化設定

```
$scope.products = [];  
$scope.currentPage = 0;  
$scope.pageSize = 5;  
    • products: 所有商品資料  
    • currentPage: 分頁當前頁碼  
    • pageSize: 每頁顯示幾筆資料  
$scope.form = {  
    id: -1,  
    category: "",  
    description: "",  
    price: "",  
    images: "",  
    title: "",  
    rating: {}  
};  
    • form: 用於新增/編輯產品的表單  
    • 當 id = -1 表示為新增操作
```

■ 分頁功能

顯示目前頁的產品：

```
$scope.pagedProducts = function () {  
    var start = $scope.currentPage * $scope.pageSize;  
    return $scope.products.slice(start, start + $scope.pageSize);  
};
```

計算總頁數：

```
$scope.numberOfPages = function () {  
    return Math.ceil($scope.products.length / $scope.pageSize);  
};
```

上一頁 / 下一頁控制：

```
$scope.prevPage = function () {  
    if ($scope.currentPage > 0) {  
        $scope.currentPage--;  
    }  
};
```

```
$scope.nextPage = function () {  
    if ($scope.currentPage < $scope.numberOfPages() - 1) {  
        $scope.currentPage++;  
    }  
};
```

```
}  
};
```

新增/編輯產品 (POST/PUT)

```
$scope.submitProduct = function () {  
    var method = "";  
    var url = "";  
  
    if ($scope.form.id == -1) {  
        method = "POST";  
        url = 'https://fakestoreapi.com/products';  
    } else {  
        method = "PUT";  
        url = 'https://fakestoreapi.com/products/' + $scope.form.id;  
    }  
  
    $http({  
        method: method,  
        url: url,  
        data: angular.toJson($scope.form),  
        headers: {  
            'Content-Type': 'application/json'  
        }  
    }).then(_success, _error);  
};
```

- 若為新增則呼叫 POST API
 - 若為修改則呼叫 PUT API (需提供 ID)
-

刪除產品 (DELETE)

```
$scope.removeProduct = function (product) {  
    $http({  
        method: 'DELETE',  
        url: 'https://fakestoreapi.com/products/' + product.id  
    }).then(_success, _error);  
};
```

- 根據 product.id 呼叫 DELETE API
-

編輯產品 (表單填入現有資料)

```

$scope.editProduct = function (product) {
    $scope.form.category = product.category;
    $scope.form.title = product.title;
    $scope.form.description = product.description;
    $scope.form.price = product.price;
    $scope.form.rating = product.rating;
    $scope.form.image = product.image;
    $scope.form.id = product.id;
};

```

- 將選取的產品內容填入 `$scope.form`，以供修改
-

重新讀取資料

```

function _refreshPageData() {
    $http({
        method: 'GET',
        url: 'https://fakestoreapi.com/products'
    }).then(function successCallback(response) {
        $scope.products = response.data;
    }, function errorCallback(response) {
        console.log(response.statusText);
    });
}

```

- 呼叫 FakeStore API 取得所有產品並存入 `$scope.products`
-

☒ 成功與錯誤處理函式

```

function _success(response) {
    _refreshPageData();
    _clearForm();
}

```

```

function _error(response) {
    console.log(response.statusText);
}

```

- `_success`: 資料更新成功後重新載入 + 清除表單
 - `_error`: 錯誤時輸出訊息至 `console`
-

清空表單資料

```

function _clearForm() {

```

```
$scope.form.category = "";
$scope.form.title = "";
$scope.form.description = "";
$scope.form.price = "";
$scope.form.rating = "";
$scope.form.image = "";
$scope.form.id = -1;
};
```

教學總結：

功能	方法	API 路徑
查詢全部產品	GET	/products
新增產品	POST	/products
編輯產品	PUT	/products/{id}
刪除產品	DELETE	/products/{id}

這個 **HTML 搭配 AngularJS 的前端畫面設計**，和你提供的 `main.js` 控制器程式碼（`fetchProducts`）是完整搭配的。以下是這份 HTML 頁面的詳細教學手冊式解說，幫助你（或學生）清楚了解整體架構與各部分功能：

 **檔案名稱：** `index.html`

頁面目標：

- 顯示產品列表（含分頁）
 - 提供新增/修改/刪除產品的表單
 - 使用 AngularJS 的 `$http` 呼叫 [Fake Store API](#)
-

 **主要引入的資源**

```
<script
src="http://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min
.js"></script>
<script src="./js/main.js"></script>
```

- **AngularJS v1.8.2**：負責雙向綁定、MVC 結構等
 - **main.js**：你自定的 AngularJS 控制器邏輯，包含讀取/新增/編輯/刪除資料的功能
-

基本樣式設定 (CSS)

```
.button { cursor: pointer; color: blue; }
td, th { border: 1px solid gray; width: 25%; text-align: left; }
table { width: 800px; }
```

簡單的按鈕風格與表格樣式設定。

AngularJS 應用與控制器啟用

```
<body ng-app="ProductManagement"
ng-controller="ProductManagementController">
```

- **ng-app**：宣告 AngularJS 應用模組名稱為 ProductManagement
 - **ng-controller**：將此區塊的資料與邏輯交由 ProductManagementController 控制
-

商品輸入表單 (新增/編輯)

```
<form ng-submit="submitProduct()">
```

當按下「Submit」按鈕時，會觸發 submitProduct() 方法(位於 main.js 裡面)。

表單欄位綁定

每個欄位都有對應 `ng-model` 綁定 `$scope.form` 物件：

欄位名稱	<code>ng-model</code> 綁定
Category	<code>form.category</code>
Title	<code>form.title</code>
Description	<code>form.description</code>
Price	<code>form.price</code>
Image Path	<code>form.image</code>

顯示產品清單（含圖片與操作）

```
<tr ng-repeat="product in pagedProducts()">
```

- `pagedProducts()`：來自 `Controller`，用來處理目前頁面要顯示的商品（有分頁機制）

顯示欄位：

欄位	資料來源
Title	<code>{{ product.title }}</code>
Price	<code>{{ product.price }}</code>
Image	<code></code>
Actions	Edit / Remove

操作按鈕：

```
<a ng-click="editProduct(product)">Edit</a>
<a ng-click="removeProduct(product)">Remove</a>
```

分頁控制區

```
<button ng-click="prevPage()" ng-disabled="currentPage == 0">Previous</button>
```

```
Page {{currentPage + 1}} of {{numberOfPages()}}  
<button ng-click="nextPage()" ng-disabled="currentPage >=  
numberOfPages() - 1">Next</button>
```

- 透過 `prevPage()` / `nextPage()` 切換頁面
 - 顯示當前頁碼與總頁數
-

☒ 總結：整體流程圖

表單輸入 → `submitProduct()`

- ↳ 新增：POST
- ↳ 編輯：PUT
- ↳ 成功後刷新資料

清單顯示 → GET 資料 (`pagedProducts` + `ng-repeat`)

刪除按鈕 → `removeProduct(product)` → DELETE

編輯按鈕 → `editProduct(product)` → 將資料填入 form

分頁 → `prevPage()` / `nextPage()`

main.js

```
function fetchProducts($scope, $http) {
    //Initialize page with default data which is blank in this example
    $scope.products = [];
    $scope.currentPage = 0;
    $scope.pageSize = 5;
    $scope.form = {
        id: -1,
        category: "",
        description: "",
        price: "",
        images: "",
        price: "",
        title: "",
        rating: {}
    };
    //Now load the data from server
    _refreshPageData();

    $scope.pagedProducts = function () {
        var start = $scope.currentPage * $scope.pageSize;
        return $scope.products.slice(start, start + $scope.pageSize);
    };

    $scope.numberOfPages = function () {
        return Math.ceil($scope.products.length / $scope.pageSize);
    };

    $scope.prevPage = function () {
        if ($scope.currentPage > 0) {
            $scope.currentPage--;
        }
    };

    $scope.nextPage = function () {
        if ($scope.currentPage < $scope.numberOfPages() - 1) {
            $scope.currentPage++;
        }
    };
}
```

```
    }  
};
```

//HTTP POST/PUT methods for add/edit employee

```
$scope.submitProduct = function () {  
  
    var method = "";  
    var url = "";  
    if ($scope.form.id == -1) {  
        //Id is absent so add employee - POST operation  
        method = "POST";  
        url = 'https://fakestoreapi.com/products';  
    } else {  
        //If Id is present, it's edit operation - PUT operation  
        method = "PUT";  
        url = 'https://fakestoreapi.com/products/' + $scope.form.id;  
    }  
  
    $http({  
        method: method,  
        url: url,  
        data: angular.toJson($scope.form),  
        headers: {  
            'Content-Type': 'application/json'  
        }  
    }).then(_success, _error);  
};
```

//HTTP DELETE- delete employee by Id

```
$scope.removeProduct = function (product) {  
    $http({  
        method: 'DELETE',  
        url: 'https://fakestoreapi.com/products/' + product.id  
    }).then(_success, _error);  
};
```

//In case of edit employee, populate form with employee data

```
$scope.editProduct = function (product) {  
    $scope.form.category = product.category;  
    $scope.form.title = product.title;  
    $scope.form.description = product.description;  
    $scope.form.price = product.price;  
    $scope.form.rating = product.rating;  
    $scope.form.image = product.image;  
    $scope.form.id = product.id;  
};
```

/* Private Methods */

//HTTP GET- get all employees collection

```
function _refreshPageData() {  
    $http({  
        method: 'GET',  
        url: 'https://fakestoreapi.com/products'  
    }).then(function successCallback(response) {  
        $scope.products = response.data;  
    }, function errorCallback(response) {  
        console.log(response.statusText);  
    });  
}
```

```
function _success(response) {  
    _refreshPageData();  
    _clearForm();  
}
```

```
function _error(response) {  
    console.log(response.statusText);  
}
```

//Clear the form

```
function _clearForm() {  
    $scope.form.category = "";  
    $scope.form.title = "";
```

```

    $scope.form.description = "";
    $scope.form.price = "";
    $scope.form.rating = "";
    $scope.form.image = "";
    $scope.form.id = -1;
  };
}

var app = angular.module("ProductManagement", []);
//Controller Part
app.controller("ProductManagementController", fetchProducts);

```

fakeproducts.html

```

<!doctype html>
<html>

<head>
  <meta charset="utf-8" />
  <title>
    AngularJS - fake product Demo using $http service
  </title>
  <script
src="http://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>
  <!-- Load AngularJS -->
  <script src="./js/main.js"></script>
  <style>
    .button {
      cursor: pointer;
      color: blue;
    }

    td,
    th {
      border: 1px solid gray;
      width: 25%;
      text-align: left;
    }

```

```
table {  
    width: 800px;  
}  
</style>
```

```
<head>
```

```
<body ng-app="ProductManagement"  
ng-controller="ProductManagementController">  
    <h1>  
        AngularJS - Use $http to invoke Product APIs  
    </h1>  
    <h2>Add/Edit Product</h2>  
    <form ng-submit="submitProduct()">  
        <table>  
            <tr>  
                <td>Category</td>  
                <td><input type="text" ng-model="form.category" size="60" /></td>  
            </tr>  
            <tr>  
                <td>Title</td>  
                <td><input type="text" ng-model="form.title" size="100" /></td>  
            </tr>  
            <tr>  
                <td>Description</td>  
                <td><input type="text" ng-model="form.description" size="100" /></td>  
            </tr>  
            <tr>  
                <td>price</td>  
                <td><input type="text" ng-model="form.price" size="60" /></td>  
            </tr>  
            <tr>  
                <td>Image Path</td>  
                <td><input type="text" ng-model="form.image" size="60" /></td>  
            </tr>  
            <tr>  
                <td colspan="2"><input type="submit" value="Submit" /></td>
```

```

        </tr>
    </table>
</form>
<table>
    <tr>
        <th>Title</th>
        <th>Price</th>
        <th>Image</th>
        <th>Actions</th>
    </tr>

    <tr ng-repeat="product in pagedProducts()">
        <td>{{ product.title }}</td>
        <td>{{ product.price }}</td>
        <td></td>
        <td><a ng-click="editProduct( product )" class="button">Edit</a> | <a
ng-click="removeProduct( product )"
                class="button">Remove</a></td>
    </tr>
</table>
<div style="margin-top: 20px;">
    <button ng-click="prevPage()" ng-disabled="currentPage ==
0">Previous</button>
    Page {{currentPage + 1}} of {{numberOfPages()}}
    <button ng-click="nextPage()" ng-disabled="currentPage >= numberOfPages()
- 1">Next</button>
</div>
</body>
</html>

```